

New Adroit Scanning Architecture (NASA)

Contents

Part 1	2
Audience	2
The problem.....	2
New architecture	3
New device agent	3
New protocol drivers.....	3
Cluster operations	4
Quality values	4
Configuration.....	5
Part 2	6
nasaclasses.h file.....	6
nasadevice.h file.....	6
Typical NASA driver.....	6
Device_NASA class	7
Part 3	10
Writing a NASA Driver.....	10
Performance Counters	10
Adding Performance Counters to your project	10
Troubleshooting Performance Counters.....	12
Part 4	15
Acknowledgements	15
Revisions	15

PART 1

Audience

The intended audience is communications specialists knowledgeable in protocols, and in the hardware technologies that make data communications possible. I also assume that some terms related to the operation of Adroit such as proxies and subscriptions make sense at a simple level at least. A working knowledge of scanning is also assumed.

The problem

Currently we identify 2 short-comings in the Adroit scanning philosophy, these are largely offset by simplicity of configuration, and some automated reasoning in our drivers. However for the more advanced customer who wants total control, and the best possible performance we identify the following.

- Double-polling of data when packing in the Front-end has overlaps
- Continued polling of data that is not needed in the client at times.

We will now lay out the reasons for the problems, and then move onto a proposal in the next section. Understanding these problems is imperative to the design of a solution.

Double-polling: This occurs because the driver always maps I/O addresses that are near to each other into one large task. This happens automatically in the driver based on an optimization algorithm which tries to fetch the largest legal blocks (PDU length) of data since this is the optimal method. This causes a hitch because of a presentation problem. For instance data in an Analog tag in Adroit has 8-byte IEEE float presentation, data in a counter would have 4-byte long. If I/O addresses for these 2 types is interspersed, the driver *has* to create 2 tasks in order to return the data to Adroit, one for the IEEE presentation, and another for the 4-byte long. Nothing however stops the driver from creating one physical task for both, and converting for both presentations (this may include Booleans and Strings) in memory, saving communications bandwidth.

Continuous polling: The scanning sub-system continues to poll for data even if it is not required. The only current work-around is to create a separate device agent for each plant segment, and scan those tags from it. Then, when the data is not required, stop the device agent. This situation is quite complex, and as a system grows with time, the benefits of this type of configuration are typically eroded by people placing more un-necessary tags on mimics, which causes these scan tasks to run again when only 1 item is needed.

Limited data types: Currently the Adroit scanning subsystem is limited to scanning Boolean, Integer, Real and String[80] data types only. The new architecture will break this limitation allowing for datetime and longer strings data types to

be communicated as well. User Defined Types, discussed later, will also be supported in that the scanning subsystem.

New architecture

NASA requires changes in two areas of Adroit's scanning subsystem:

New device agent

A new Device class (called **Device_NASA**) inherits from the current Device class in the device agent to handle the new architecture.

The **Device_NASA** class does NOT require the use of scan agents. The responsibility for scheduling scan tasks are now demoted to the protocol driver as explained later.

The **Device_NASA** class implements a callback method defined by the **CAdroitProtocolDriverCallbacks** class which will allow NASA drivers to call back into the device agent.

On creation of a device agent, the decision to use the legacy device class or the new class takes place in the Device agent's construct() API call whereby the driver DLL would be tested for the existence of a new export function called **CreateAdroitProtocolDriver**, otherwise falling back to the legacy **ProtDrvCreate**. If **CreateAdroitProtocolDriver** exists, it implies that the protocol driver is compliant to the new scanning architecture and will be controlled by an instance of **Device_NASA**.

In order to cater for on demand polling, a scan flag bit mask will be introduced for each scanned item that will denote the type of scanning required for each scanned tag. These are:

- Bit 0 = enable reads
- Bit 1 = enable writes
- Bit 2 = output initialize
- Bit 3 = demand scan

It is envisaged that write-only tags will never be marked bad.

The scanned Adroit tag type is NOT passed to the driver. The driver shall return all values as COleVariant's and the **Device_NASA** class shall be responsible for converting these returned values into the required Adroit type.

The driver issues callback to the **Device_NASA** class in much the way OPC callbacks work viz an array of tag handles, an array of timestamps, an array of variant values and an array of qualities.

New protocol drivers

Unlike the legacy architecture, where flat C DLL export functions are used to interface to the protocol driver, NASA drivers will derive from an abstract class called **CNASADevice** which will contain the necessary functions for interfacing. The interface specification is included in the file `nasadevice.h` with further definitions defined in `nasaclasses.h`.

Cluster operations

Cluster scanning operations, such parallel scanning will now be controlled at the protocol driver level on a device by device basis. The **Device_NASA** class will inform the driver of the current agent server cluster status via a SetClusterStatus() method call and this method will return whether the driver is configured for shadowing or not, thus instructing the device agent to perform shadowed putslots if required.

The legacy global cluster shadowed scanning configuration will only apply to legacy architecture protocol drivers.

Quality values

Quality values will honour the defined OPC quality values. These are currently defined as:

```
// --- QUALITY MASK VALUES
#define OPC_QUALITY_MASK          0xC0
#define OPC_STATUS_MASK          0xFC
#define OPC_LIMIT_MASK           0x03
// --- QUALITY BIT VALUES
#define OPC_QUALITY_BAD           0x00    //x
#define OPC_QUALITY_UNCERTAIN    0x40
#define OPC_QUALITY_GOOD         0xC0
// --- STATUS BIT VALUES
#define OPC_QUALITY_CONFIG_ERROR  0x04
#define OPC_QUALITY_NOT_CONNECTED 0x08
#define OPC_QUALITY_DEVICE_FAILURE 0x0c    //x
#define OPC_QUALITY_SENSOR_FAILURE 0x10    //x
#define OPC_QUALITY_LAST_KNOWN    0x14
#define OPC_QUALITY_COMM_FAILURE  0x18
#define OPC_QUALITY_OUT_OF_SERVICE 0x1C    //x
#define OPC_QUALITY_LAST_USABLE   0x44    //x
#define OPC_QUALITY_SENSOR_CAL    0x50    //x
#define OPC_QUALITY_EGU_EXCEEDED  0x54    //x
#define OPC_QUALITY_SUB_NORMAL    0x58    //x
#define OPC_QUALITY_LOCAL_OVERRIDE 0xD8    //x
#define OPC_LIMIT_OK              0x00    //x
// --- LIMIT BIT VALUES
#define OPC_LIMIT_LOW              0x01    //x
#define OPC_LIMIT_HIGH             0x02    //x
#define OPC_LIMIT_CONST            0x03    //x
```

Many of the above will be unusable from within Adroit in that that they cannot be determined. Such values are marked with the trailing “//x”. Typically, tag qualities would be used as follows:

OPC_QUALITY_CONFIG_ERROR

Unknown scan address. The quality for such a tag is never changed.

OPC_QUALITY_UNCERTAIN

The protocol does not allow for reading this address i.e. write only address

OPC_QUALITY_NOTCONNECTED

Device is started but the group has not yet been enabled

OPC_QUALITY_GOOD

Device is started and group is enabled and the protocol driver retrieved a valid value for this tag at the last scan

OPC_QUALITY_LAST_KNOWN

Device started but group has been disabled

OPC_QUALITY_COMM_FAILURE

Device is started and group is enabled but the protocol driver failed to connect to the FED at the last scan

Do we need others ????????

Dnp needs

1. *Online*
2. *Restart*
3. *ComLost*
4. *RemoteForcedData*
5. *LocalForcedData*
6. *RollOver*
7. *ChatterFilter*

IEC needs

1. *Cyclic (data was sent as result of cyclic req.)*
2. *Background*
3. *Spontaneous (I.e. Event-based)*
4. *Initialized : Similar to DNP "reset"*
5. *Requested (data is result of a request)*

Configuration

The **CNASADevice** base class contains support for the definition, storage, loading and saving of any number of configuration properties required by the NASA device. Each property is defined as a **CNASAPropertyItem**.

Two lists of configuration properties, primary and secondary, are held as **CNASAPropertyItems** instances.

The base class also supports a native setup dialog containing a property editor control (CMFCPropertyGridCtrl). The contents of the property editor are defined by the **CNASAPropertyItem**'s

NASA driver developers are free to use their own setup dialog instead of using the built-in native setup dialog.

PART 2

nasaclasses.h file

The nasaclasses.h file contains the following classes:

CAdroitProtocolDriverCallbacks	Base class for Device_NASA class allowing for CNASADevice instances to call back into the device agent
CScanItem	Defines the scan configuration for an FED address to be scanned
CAdroitOleDateTime	Helper class to the COleDateTime to provide millisecond support
CNASAPropertyItem	Defines a single configuration property for CNASADevice's
CNASAPropertyItems	A collection of CNasaPropertyItems

nasadevice.h file

The nasadevice.h file contains the following classes:

CNASADevice	Base class for all NASA drivers. The new device class will contain an instance of this class as instantiated by the driver itself
CAdroitProtocolDriverCallbacks	Defines the callback that must be implemented by Adroit's Device agent to allow NASA drivers to process new values from the FED.

Typical NASA driver

A typical NASA driver derives from CNASADevice as follows:

```
class ADROITDRIVER_EXT_CLASS COPCUADevice : public CNASADevice
{
public:
    // --- runtime methods -----
    COPCUADevice(LPCTSTR szDeviceName,
                 CAdroitProtocolDriverCallbacks* hAdroitDevice);
    virtual ~COPCUADevice();
    virtual CScanItem* AddScanAddress(LPCTSTR szAddress,
                                      DWORD dwRate,
                                      DWORD dwScanFlag,
                                      double dblDeadband,
                                      BOOL bSubscribed,
                                      CString& csNextAddress,
                                      void* pDeviceAgentHandle);

    BOOL Browse(COPCUADevice* pd,
                CString& csError,
                HWND hwnd);
    virtual void DynamicConfig(void);
    virtual void EnableItem(void* pClientHandle,
```

```

        BOOL bEnable = TRUE);
void OutputScopeMsg(CString cs_text,
                    WORD colour);
virtual void RemoveItem(void* pClientHandle);
virtual void ResetConnection();
virtual void Setup();
virtual void Start(BOOL bStart = TRUE);
virtual void WriteItem(void* pClientHandle,
                       COleVariant& ovValue);
// --- configuration methods -----
virtual void SetPropertyItem(LPCTSTR szName,
                             LPCTSTR szValue);
virtual CString OnSetPropertyButton(DWORD dwData);
virtual void Unload();

void Run();

BOOL m_bThreadRunning;
BOOL m_bDeviceStarted;
long m_UseCount;           // for use by CDeviceArray()
WORD m_Status;            // status (DEVICE_OK or DEVICE_BAD)
long m_Retries;
BYTE m_TimeStampMethod;

DriverEngineStatus m_EngineState;
COPCUAConnection* m_pOPCUAConnection;

private:
    HANDLE m_hDevice;
    HANDLE m_MonFileHandle;    // monitor memory-mapped file structure
    BOOL CreateMonitor();

    // protocol dependant members
    PSECURITY_DESCRIPTOR m_pSD;    // ... and security descriptor
    CString BrowseComputers();
    CString BrowseOPCServers();
    BOOL BrowseOPCServers(CString& csServerName,
                           CString& csServerUrl,
                           EnumUserTokenType& ut,
                           EnumMessageSecurityMode& sm, CString& csSecPolicy);
    BOOL Connect(CString& csError);
};

```

Device_NASA class

The new Device_NASA class is as follows:

```

class Device_NASA : public Device, public CAdroitProtocolDriverCallbacks
{
    // --- Interface -----

public:

    // --- Interface Operations of Device_NASA class ---

    // constructor - creates Device_NASA instances
    Device_NASA (

```

```

    HMODULE,                // -> module handle
    CNASADevice*,           // -> protocol driver class instance
    AgentHandle h,          // -> agent handle
    LPCTSTR szDeviceName,   // -> agent name, e.g. BOILER1
    AgentId& agentId,       // -> agent id
    LPCTSTR workgroupName,  // -> name of workgroup containing agent
    LPCTSTR channelName,    // -> channel name for subscriptions
    BOOL& operationOk       // <- returns TRUE if successful
);

// destructor - cleans up Device_NASA instances
~Device_NASA();

virtual BOOL scanTag (
    LPCTSTR,                // -> scanning specification, e.g.
                           // "DB21:26:0:12, AN-01.rawVal, 1000, 10.0"
    BOOL = TRUE             // -> FALSE if scanned agent already joined
);                          // returns TRUE if successful

virtual BOOL removeTag (
    LPCTSTR                 // -> I/O agent "Address" slot contents, e.g.
                           // "DB21:26:0:12, AN-01.rawVal, 1000, 10.0"
);                          // returns TRUE if successful

virtual void stopMember    ();

virtual void startMember   ();

virtual BOOL attachDriver (
    CString& na             // <- next address
);

void ProcessOutput(
    TagListEntry* tlePtr,   // corresponding tag entry
    long slotType,          // slot type of tag whose value has changed
    long slotSize,          // slot size of tag whose value has changed
    void* pSource,          // value of tag whose value has changed
    long* pAgentStatus      // status of tag whose value has changed
);

// CAdroitProtocolDriverCallbacks functions
virtual void ProcessInput(CScanItem* psi, DWORD dwConnection);
virtual void ProcessInputs(CPtrArray& items, DWORD dwConnection);

// --- Implementation -----

CNASADevice* m_apdDriver;

private:

DWORD m_dwServerType;
BOOL m_bShadowed;
BYTE m_data[1000];

DWORD m_dwBulkArraySize;
TagId *m_tagIdArray;
long *m_slotNoArray;

```



```
long      *m_sizeArray;  
long      *m_typeArray;  
long      *m_statusArray;  
PVOID     *m_dataArray;  
Time      *m_timeArray;  
ServerHandle *m_handleArray;  
CStringArray m_namePtrArray;  
  
void DeviceHealthyControl();  
};
```

PART 3

Writing a NASA Driver

Performance Counters

- Performance counters are CHEAP and FAST. Besides code clutter they could be left there indefinitely, perhaps `#ifdef`'d at the risk of further code clutter etc.
- Hold their last known state, so GOOD FOR CRASH code position checks.
- Useful for activity (State engine states) and statistical information.
- Operate on a simple polling mechanism, `openfunc` , (`collectfunc` in a loop) , `closefunc` when finished you will see these exports in the def.

Adding Performance Counters to your project

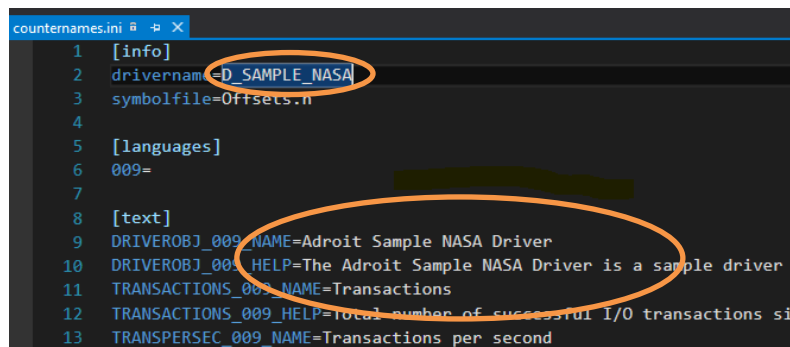
You will only need to slightly modify the following files in your project: (Change the descriptions where necessary – if you intend to add or remove counters please be careful , modify the indexes appropriately and TEST it , `user1-user16` are made available without need of adjustment one merely has to document what each count represents and rename them for `perfmon`.)

Make sure you have the following in your project:

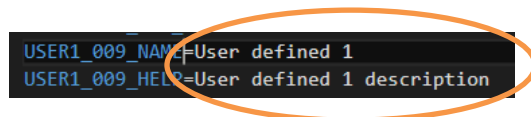
1. Counternames.ini
2. Perfmon.h
3. Perfmon.cpp
4. Offsets.h
5. Add `OpenFunc`, `CloseFunc`, `CollectFunc` (and `ProtDrvGetCounterNames`, if required) in your def file

Make sure you delete or make writeable `counternames.inc` and `offsets.inc`

Edit the `counternames.in` file to reflect the correct information including driver name and description details in the various counters, and names for the `user1` through `user16` if you are going to use these in your code, this name will be visible to the `perfmon` system. Save.



```
counternames.ini
1 [info]
2 drivernam=D_SAMPLE_NASA
3 symbolfile=Offsets.h
4
5 [languages]
6 009=
7
8 [text]
9 DRIVEROBJ_009_NAME=Adroit Sample NASA Driver
10 DRIVEROBJ_009_HELP=The Adroit Sample NASA Driver is a sample driver
11 TRANSACTIONS_009_NAME=Transactions
12 TRANSACTIONS_009_HELP=Total number of successful I/O transactions si
13 TRANSPERSEC_009_NAME=Transactions per second
```



```
USER1_009_NAME=User defined 1
USER1_009_HELP=User defined 1 description
```

If you are using custom counters (User1 to User16), add ProtDrvGetCounterNames function to your perfmon.cpp file to match the names in your counternames.ini file. This will allow the protocol driver monitor 2 to display your user counters.

Matching perfmon changes will be done in the driver project as required, these are most important, the pdrvmon changes are optional...

I recommend you rename the counter names for pdrvmon to match the name in the counternames.ini do not exceed 16 characters (upto the "=" below) as this could have an impact as how the pdrvmon window is sized. .

```
__declspec (dllexport)void ProtDrvGetCounterNames(CStringArray &csa)
{
    // --- next line is compulsory to get the basic 11 counters
    CNASADriver::GetCounterNames(csa);
    // --- uncomment only the counters below you are using - up to 16 counters
    //     the names should be the same as in your counternames.ini file
    /*
    csa.Add("User 1          = %9d");
    */
}
```

The call to CNASADriver::GetCounterNames above can be replaced with your own code as follows:

```
csa.Add("Driver object");           // compulsory
csa.Add("Transactions = %9d");      // compulsory
csa.Add("Transactions/s = %9.3f");  // system
csa.Add("Percent errors = %9.3f");  // system
csa.Add("Init failures = %9d");      // compulsory
csa.Add("Retries = %9d");            // optional
csa.Add("Port switches = %9d");      // optional
csa.Add("Protocol errors = %9d");    // optional
csa.Add("Read errors = %9d");        // compulsory
csa.Add("Write errors = %9d");       // compulsory
csa.Add("Read timeouts = %9d");      // optional
csa.Add("Write timeouts = %9d");     // optional
```

The compulsory counters above must be used by your code.

If you are not using some of the optional counters (e.g. port switches and protocol errors) above you could modify the above code to read:

```
csa.Add("Driver object");           // compulsory
csa.Add("Transactions = %9d");      // system
csa.Add("Transactions/s = %9.3f");  // system
csa.Add("Percent errors = %9.3f");  // compulsory
csa.Add("Init failures = %9d");      // compulsory
csa.Add("Retries = %9d");            // optional
csa.Add("- = %9d");                  // optional
csa.Add("- = %9d");                  // optional
csa.Add("Read errors = %9d");        // compulsory
csa.Add("Write errors = %9d");       // compulsory
csa.Add("Read timeouts = %9d");      // optional
csa.Add("Write timeouts = %9d");     // optional
```

This will cause a compacted display of counters in the protocol driver monitor

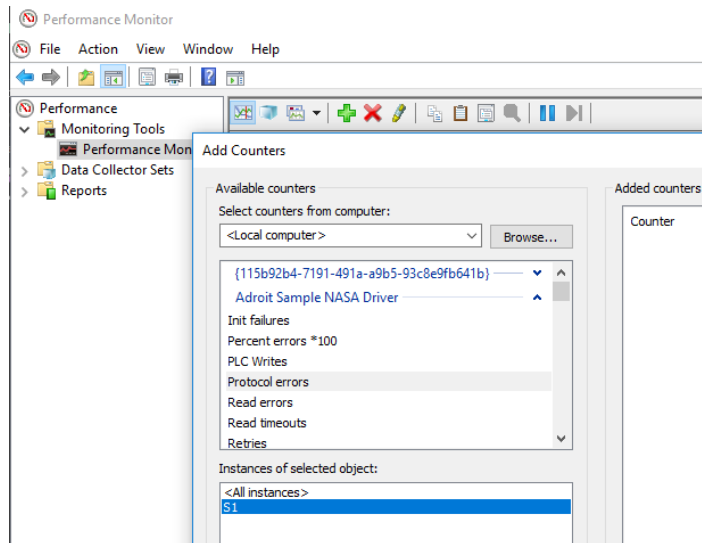
Make sure you test this change by running the protocol driver monitor (pdrvmon.exe) under option 2.

To allow performance monitor to operate correctly, run an administrative cmd console window , navigate to your driver source path and run the following:

`"..\incs.bat"`

Check that counternames.inc and offsets.inc have been rebuilt.

You should be good to go, rebuild the driver dll, copy to agent path, restart the agent server attach a device , see if the driver and its device instance is visible in the perfmon list.



Troubleshooting Performance Counters

To test if performance counters are active, goto permon.cpp line ~456 and change the perfversion, this tells the setup procedure to reinstall performance counters, this is especially important if performance counters are ADDED even if it is just a name change when including the use of a new "USERx" count.

Perfmon.cpp:

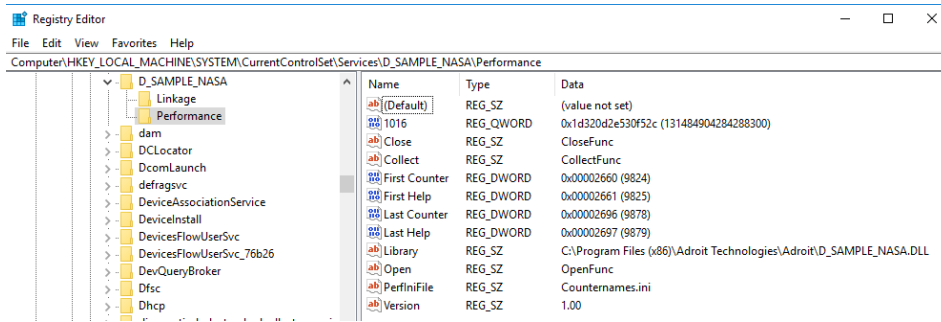
```
CHAR perfversion[] = "1.00";
```

After compile rerun setup.

Any name change you must rerun incs.bat, and you cannot transfer incs between projects.

Another check is via regedit:

HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\D_SAMPLE_NASA



Make sure there is no disabled key here.

Make sure you see First counter to Last help

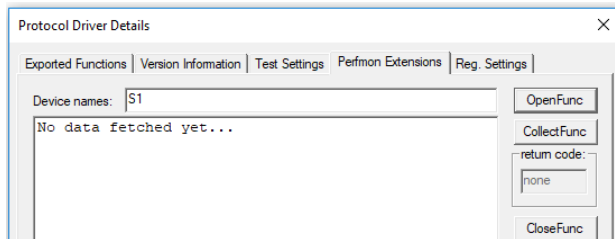
Make sure you dll path is correct

You can change the version here which has the same effect as changing it in the dll (perfversion)

You can delete the whole reg key, remember to run setup and click ok again.

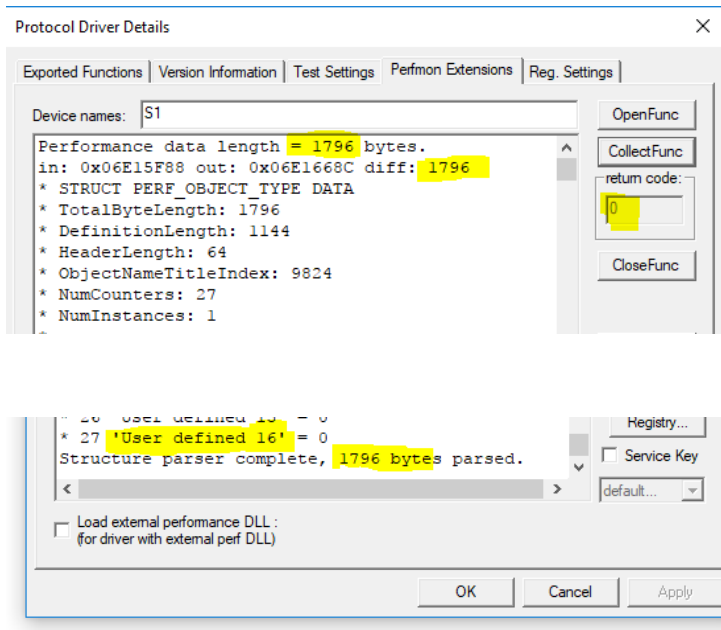
The linkage key records the device instances, it should hold the name of your device.

Using testpdv.exe (it now partially support nasa drivers) , load the nasa driver , the click on tools
-> perfmon extensions , you will see this :



You must see the device name(s) , depends on the linkage key. Keep an eye on the return code , when you click open collect and close.

Clicking open , return code must show opened, close -> closed, collect ->0.



The data length and diff and structure parser (At the bottom) must match, browse down, errors are usually described at the bottom of the list.

The counternames as defined in counternames.ini(inc) are displayed near the bottom.

PART 4

ACKNOWLEDGEMENTS

Author: Mike Lamusse mikel@adoit.co.za

Assisted by: David Jordaan davidj@adroit.co.za and Carl Swanepoel carls@adroit.co.za.

REVISIONS

Date	author	Changes
Mar 2016	MGL	Initial version
Mar 2017	MGL	As built
30 Aug 2017	DJ	Added perfmon details.